Day 33 Task – Password Security & Authentication Analysis

Password security and authentication are essential components of cybersecurity. Web applications use authentication systems to verify user identities and prevent unauthorized access to personal accounts and sensitive information.

Weak passwords, improper password storage, and repeated login attempts can expose applications to threats such as brute-force attacks, password guessing, and account compromise.

The Password Security & Authentication Analysis project is an educational web application developed using Python, Flask, SQLite, HTML, CSS, and JavaScript. It demonstrates secure password handling, password strength evaluation, user authentication, account lockout, and authentication activity logging.

The project focuses on understanding how authentication systems protect user accounts through multiple security mechanisms.

Objective

The main objective is to develop a basic authentication application that demonstrates secure password management and defensive authentication practices.

The project aims to:

- Implement user registration and login functionality.
- Store passwords securely using password hashing.
- Evaluate password strength and enforce password policies.
- Track unsuccessful login attempts.
- Implement temporary account lockout.
- Record authentication-related activities.
- Understand common authentication vulnerabilities and their prevention.

Tools and Technologies Used

Python: Used for backend programming and authentication logic.

Flask: Used to create the web application, manage routes, process user requests, and handle authentication sessions.

HTML: Used to structure the registration, login, homepage, and dashboard pages.

CSS: Used to design the application's interface.

JavaScript: Used to provide real-time password strength feedback.

SQLite: Used to store user account information, password hashes, failed login counts, lockout details, and authentication events.

Werkzeug: Used for secure password hashing and password verification through its built-in security utilities.

Git and GitHub: Used for source code management and version control.

Python Logging: Used to record authentication-related activities.

Project Architecture

The application follows a simple client-server architecture consisting of three main layers.

Frontend Layer: The frontend uses HTML, CSS, and JavaScript to provide user registration, login, password strength feedback, and dashboard interfaces.

Backend Layer: Flask handles user requests, validates inputs, verifies passwords, manages account lockout, and controls user sessions.

Database Layer: SQLite stores registered user information, password hashes, failed authentication counts, lockout information, and authentication records.

The frontend communicates with the Flask backend, which processes the requests and interacts with the database to perform authentication operations.

User Registration and Login

The registration module allows users to create accounts using a unique username and password. During registration, the application validates the username and checks whether the password satisfies the defined security policy.

The login module verifies the submitted credentials against the registered account information. When a valid username and password are provided, the application authenticates the user and provides access to the dashboard.

Incorrect credentials are rejected, and the failed login attempt is recorded.

Password Strength Checker

The application includes a password strength checker implemented using JavaScript.

It evaluates passwords based on the following conditions:

- Minimum length of eight characters.
- At least one uppercase letter.
- At least one lowercase letter.
- At least one number.
- At least one special character.

A visual strength indicator provides feedback as the user enters a password.

Password requirements are also validated on the Flask backend because client-side JavaScript alone cannot enforce security policies.

Secure Password Hashing

Secure password storage is an important feature of this project.

Storing passwords in plaintext creates a serious security risk because anyone who gains access to the database could view the original credentials.

To prevent this, the application uses Werkzeug's password hashing utilities with the scrypt algorithm.

The generate_password_hash() function generates a password hash during registration. The check_password_hash() function verifies a submitted password during login.

The original password is not stored directly in the database. This helps protect user credentials even if the stored account information is exposed.

Failed Login Tracking

The application includes a failed-login tracking mechanism to monitor unsuccessful authentication attempts.

Whenever a user enters an incorrect password, the application increases the failed-attempt counter associated with that account.

Repeated unsuccessful attempts can indicate password guessing or a brute-force attempt. Tracking these events helps the application identify repeated authentication failures and apply protective measures.

Account Lockout Mechanism

A temporary account lockout mechanism provides additional protection against repeated incorrect login attempts.

The configured policy is:

- Five consecutive failed login attempts trigger an account lockout.
- The account remains locked for ten minutes.
- Login attempts during the lockout period are rejected.
- The failed-attempt counter is reset after the lockout expires.
- A successful login resets the failed-attempt counter.

This feature demonstrates how authentication systems can limit repeated password-guessing attempts.

Account lockout must be carefully configured because attackers could otherwise deliberately lock legitimate users out of their accounts.

Authentication Logging

Authentication logging records important account-related activities to support monitoring and security investigation.

The application records events such as:

- Successful user registration.
- Successful login.

- Failed login attempts.
- Account lockout.
- Login attempts during an active lockout.
- User logout.

The application maintains authentication records in the SQLite database and an auth.log file.

These logs help identify repeated login failures, investigate suspicious activity, and understand user authentication behavior.

Sensitive information such as submitted passwords should never be written to authentication logs.

Database Management Using SQLite

SQLite is used as the database management system for the application. It provides a lightweight way to store and retrieve account information and authentication records.

The database contains two primary tables.

Users Table: Stores the unique user ID, username, password hash, failed login count, lockout expiry information, and account creation timestamp.

Authentication Logs Table: Stores the event ID, username, event type, source IP address, additional details, and event timestamp.

This structure allows the application to maintain account information separately from authentication activity.

Testing and Validation

The following test cases can be used to evaluate the application's security features.

Test Case	**Expected Result**
Register with valid details	Account should be created
Register with an existing username	Registration should be rejected
Enter a weak password	Password policy validation should reject it
Login with correct credentials	User should access the dashboard
Login with incorrect credentials	Failed attempt should be recorded
Enter incorrect password five times	Temporary account lockout should activate
Login during lockout	Login should be rejected
Login successfully after lockout expires	Account should become accessible

Review authentication logs	Relevant login events should be recorded

These test cases provide a basis for verifying that the authentication and security mechanisms behave as intended.

Security Recommendations

The following practices can further strengthen the application:

- Never store plaintext passwords.
- Use established password-hashing libraries.
- Apply strong password policies.
- Use multifactor authentication where possible.
- Limit repeated failed login attempts.
- Protect authentication logs from unauthorized modification.
- Use secure session management.
- Restrict database and administrative access.
- Use HTTPS when deploying the application.
- Protect production secret keys and sensitive configuration values.
- Test authentication and recovery mechanisms regularly.

Limitations and Future Enhancements

The current application is an educational prototype focused on basic authentication security. It can be improved with additional features such as email verification, secure password recovery, multifactor authentication, administrator dashboards, role-based access control, and advanced login monitoring.

Further improvements could include stronger rate limiting, production-grade CSRF protection, secure session configuration, and deployment using HTTPS with a production WSGI server.

These improvements would make the application more suitable for real-world security requirements.

The Password Security & Authentication Analysis project demonstrates how fundamental cybersecurity concepts can be implemented through a functional web application.

By combining Python, Flask, SQLite, HTML, CSS, JavaScript, and Werkzeug, the application demonstrates user registration, secure password hashing, password strength evaluation, login authentication, failed-login tracking, temporary account lockout, and authentication logging.

This project provides practical knowledge of defensive authentication techniques and highlights the importance of protecting user credentials, managing account access, and monitoring authentication events.

It also demonstrates how multiple security controls work together to create a safer authentication environment.